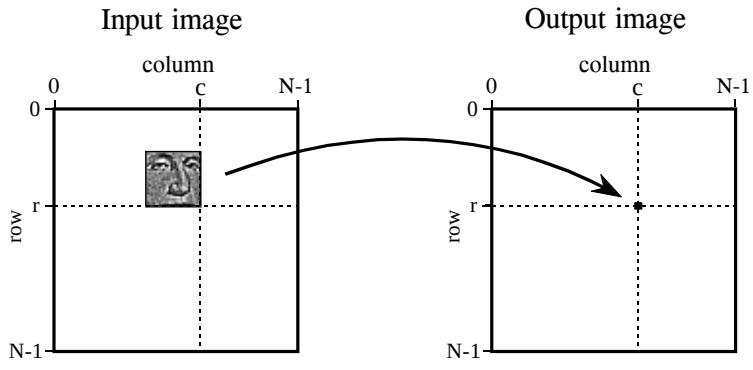

FIGURE 24-15
Image convolution viewed from the output side. Each pixel in the output signal is equal to the sum of the pixels in the rotated PSF multiplied by the corresponding pixels in the input image.

FFT convolution provides the same output image as the conventional convolution program of Table 24-1. Is the reduced execution time provided by FFT convolution really worth the additional program complexity? Let's take a closer look. Figure 24-16 shows an execution time comparison between conventional convolution using *floating point* (labeled FP), conventional convolution using *integers* (labeled INT), and FFT convolution using floating point (labeled FFT). Data for two different image sizes are presented, 512×512 and 128×128.

First, notice that the execution time required for FFT convolution does not depend on the size of the kernel, resulting in flat lines in this graph. On a 100 MHz Pentium personal computer, a 128×128 image can be convolved

```
100 CONVENTIONAL IMAGE CONVOLUTION
110 '
120 DIM X[99,99]                    'holds the input image,   100×100 pixels
130 DIM H[28,28]                    'holds the filter kernel,   29×29 pixels
140 DIM Y[127,127]                  'holds the output image, 128×128 pixels
150 '
160 FOR R% = 0 TO 127               'loop through each row and column in the output
170 FOR C% = 0 TO 127               'image calculating the pixel value via Eq. 24-3
180 '
190   Y[R%,C%] = 0                  'zero the pixel so it can be used as an accumulator
200 '
210   FOR J% = 0 TO 28              'multiply each pixel in the kernel by the corresponding
220   FOR K% = 0 TO 28              'pixel in the input image, and add to the accumulator
230     Y[R%,C%] = Y[R%,C%] + H[J%,K%] * X[R%-J%,C%-J%]
240   NEXT K%
250   NEXT J%
260 '
270 NEXT C%
280 NEXT R%
290 '
300 END
```

TABLE 24-1